

Q34. Coin Change vs Knapsack Problem (DP Comparison)

1. Aim

To solve the Coin Change and 0/1 Knapsack problems using Dynamic Programming, compare their recurrence relations and table structures, and analyze their constraints, complexity, and scalability.

2. Problem Statement

Coin Change and Knapsack are optimization problems that can be solved using Dynamic Programming. Coin Change finds the minimum number of coins required to make a given amount, while Knapsack finds the maximum value that can be obtained within a given weight capacity.

3. Objectives

- Solve Coin Change using Dynamic Programming.
- Solve 0/1 Knapsack using Dynamic Programming.
- Compare their recurrence relations and DP tables.
- Analyze the constraints of both problems.
- Evaluate time and space complexity.
- Understand the scalability of Dynamic Programming solutions.

4. Input Details

Coin Change

- Coins = [1, 2, 5]
- Target Amount = 11
- Each coin can be used multiple times.

Knapsack

- Weights = [2, 3, 4, 5]
- Values = [3, 4, 5, 6]

- Capacity = 5
- Each item can be selected only once.

5. Algorithm Used

Dynamic Programming (DP)

Two DP approaches are used:

1. **Coin Change DP** – Finds the minimum number of coins needed for a target amount.
2. **0/1 Knapsack DP** – Finds the maximum value that can be obtained within a weight capacity.

6. Algorithm

Coin Change

1. Create a DP array from 0 to the target amount.
2. Set $dp[0] = 0$.
3. For each amount, check every available coin.
4. If the coin can be used, update the minimum number of coins.
5. Continue until the target amount is reached.
6. Return the minimum number of coins.

Recurrence:

$$dp[amount] = \min(dp[amount], dp[amount - coin] + 1)$$

0/1 Knapsack

1. Create a DP table for items and capacities.
2. Initialize the table with zero.
3. For each item, check whether it can fit into the current capacity.
4. If it fits, compare including and excluding the item.
5. Store the maximum value.
6. Return the value at the final capacity.

Recurrence:

$$dp[i][w] = \max(dp[i-1][w], \text{value}[i-1] + dp[i-1][w - \text{weight}[i-1]])$$

PROGRAM:

```
1 def coin_change(coins, amount):
2     dp = [float('inf')] * (amount + 1)
3     dp[0] = 0
4
5     for i in range(1, amount + 1):
6         for coin in coins:
7             if coin <= i:
8                 dp[i] = min(dp[i], dp[i - coin] + 1)
9
10    return dp[amount]
11
12
13 # 0/1 Knapsack using Dynamic Programming
14
15 def knapsack(weights, values, capacity):
16     n = len(weights)
17
18     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
19
20     for i in range(1, n + 1):
21         for w in range(capacity + 1):
22
23             if weights[i - 1] <= w:
24                 dp[i][w] = max(
25                     dp[i - 1][w],
```

OUTPUT:

```
Coin Change Problem
-----
Coins: [1, 2, 5]
Target Amount: 11
Minimum Coins Required: 3

0/1 Knapsack Problem
-----
Weights: [2, 3, 4, 5]
Values: [3, 4, 5, 6]
Capacity: 5
Maximum Value: 7
```

9. Comparison of Recurrence Relations

Feature	Coin Change	0/1 Knapsack
Goal	Minimum number of coins	Maximum value

DP State	Amount	Item and capacity
Choice	Select a coin	Include or exclude an item
Repetition	Coins can be reused	Items cannot be reused
Main Operation	Minimum	Maximum

10. DP Table Structure

Coin Change

Coin Change uses a **one-dimensional DP array**:

`dp[0], dp[1], dp[2], ... dp[amount]`

Each position represents the minimum number of coins required for that amount.

Knapsack

Knapsack normally uses a **two-dimensional DP table**:

`dp[item][capacity]`

Each cell represents the maximum value possible using the available items for a particular capacity.

11. Constraint Differences

Coin Change

The main constraint is the **target amount** and the available coin denominations. Coins can generally be used multiple times.

Knapsack

The main constraints are the **number of items, item weights, item values, and maximum capacity**. In the 0/1 version, each item can be selected at most once.

12. Computational Complexity

Coin Change

If there are n coin types and the target amount is A :

Time Complexity: $O(n \times A)$

Space Complexity: $O(A)$

0/1 Knapsack

If there are n items and capacity is W :

Time Complexity: $O(n \times W)$

Space Complexity: $O(n \times W)$

13. Scalability

Dynamic Programming works efficiently for moderate input sizes because it avoids solving the same subproblems repeatedly.

However, both algorithms become expensive when the target amount or knapsack capacity becomes very large. Knapsack can also require significant memory because of its two-dimensional table.

For large inputs, memory-optimized DP or other optimization techniques can be considered.

14. Analysis

The two problems use the same basic Dynamic Programming principle but model their states differently.

Coin Change focuses on reaching a target amount with the minimum number of coins. Since coins can be reused, the DP state mainly depends on the current amount.

Knapsack considers both the available items and remaining capacity. Each item creates a choice between including and excluding it, so a two-dimensional state is commonly used.

15. DP Modeling Insights

- The most important step is defining the correct **DP state**.
- The recurrence should represent the available choices.
- Coin Change uses a minimum operation.
- Knapsack uses a maximum operation.
- Constraints determine the size of the DP table.

- Identifying overlapping subproblems helps avoid repeated calculations.

16. Result

The Coin Change problem produced a minimum of **3 coins** to make amount **11**. The 0/1 Knapsack problem produced a maximum value of **7** within capacity **5**.

Both problems were successfully solved using Dynamic Programming, demonstrating how different problem constraints require different DP states and recurrence relations.

17. Conclusion

Coin Change and 0/1 Knapsack demonstrate how Dynamic Programming can solve optimization problems efficiently by storing previously calculated results. Coin Change uses a one-dimensional table based on the target amount, while Knapsack generally uses a two-dimensional table based on items and capacity. The choice of DP model depends on the problem constraints, objective, and allowed choices.